ScholarPath
System Architecture Deep Dive

Clean Architecture · CQRS with MediatR · Azure Communication Services Video Meetings · Analytics Data Pipeline · Real-time SignalR Hubs. A complete reference for the academic defence committee.

Backend: .NET 10 · ASP.NET Core · EF Core 10 · MediatR 14 · Hangfire · SignalR · Stripe · Azure OpenAI / ACS
Frontend: React 19 · Vite 8 · TypeScript 6 · Tailwind v4 · TanStack Query · i18next (EN+AR full RTL)
Data: SQL Server 2022 · Redis · Azure ADF · dbt · Synapse · Power BI · Stream Analytics

Contents

1. System Overview

ScholarPath is a **gated bilingual (EN + AR / RTL) scholarship management platform** serving students, consultants, companies, and administrators. The system is organized into **18 Product Backlog epics** (PB-001..PB-018), covering the full lifecycle from account creation to data warehousing.

PB	Module	Owner	PB	Module
001	Auth & Onboarding	@Madiha	010	Notifications
002	Profile & Account	@Madiha	011	Admin Portal
003	Scholarship Discovery	@Norra	012	Audit & Compliance
004	Application Tracking	@Norra	013	Payment Processing
005	Company Review / Payment	@Yousra	014	Profit Share
006	Consultant Booking	@Tasneem	015	Analytics Foundation
007	Community + Chat	@Madiha	016	Data Warehouse
008	AI Features	@Mahmoud	017	AI Economy
009	Resources Hub	@Madiha	018	Real-time Streaming

Key Point: Numbers

211 functional requirements · 203 use cases · 159 user stories · every FRs and USs tracked in `docs/SRS.md` and `.specify/specs/PB-xxx/`.

2. Clean Architecture

Concept: Clean Architecture

Clean Architecture organises code into concentric layers where **dependencies only point inward**: the Domain layer knows nothing about the database, the Framework, or the UI. This makes the business logic independently testable and portable.

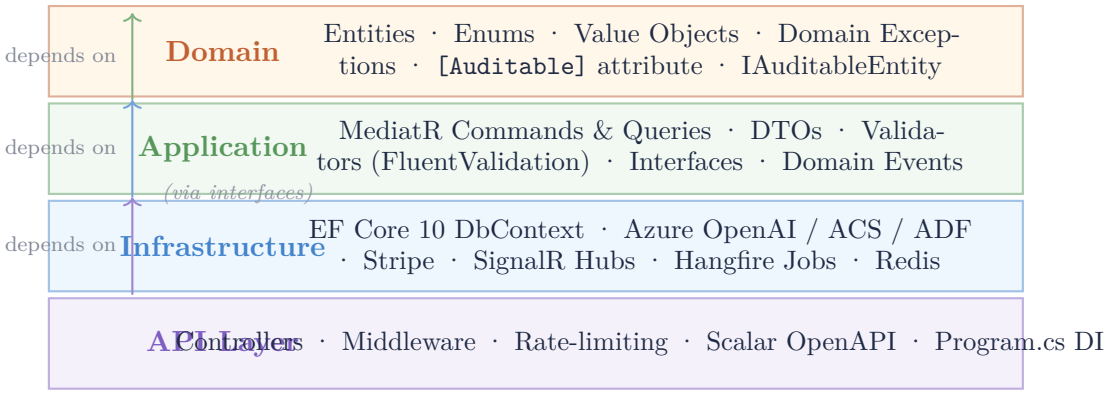

Figure 1. Clean Architecture layers in ScholarPath. Dependencies point inward only — Infrastructure depends on Application interfaces (not vice versa), enforced by separate C# project references.

Layer	Project	Key contents
Domain	`ScholarPath.Domain`	40+ EF entities, enums, domain events
Application	`ScholarPath.Application`	100+ MediatR handlers, DTOs
Infrastructure	`ScholarPath.Infrastructure`	EF DbContext, all external service impls
API	`ScholarPath.API`	32 controllers, Program.cs, middleware

3. CQRS with MediatR

Concept: CQRS

Command Query Responsibility Segregation separates every operation into one of two categories:

Command — changes state (write). Returns void or a minimal result. Example: `AcceptBookingCommand`, `GenerateRecommendationsCommand`.

Query — reads state without any side-effect. Example: `GetMyRecommendationsQuery`, `GetBookingRecordingsQuery`.

3.1 How Every Request Flows

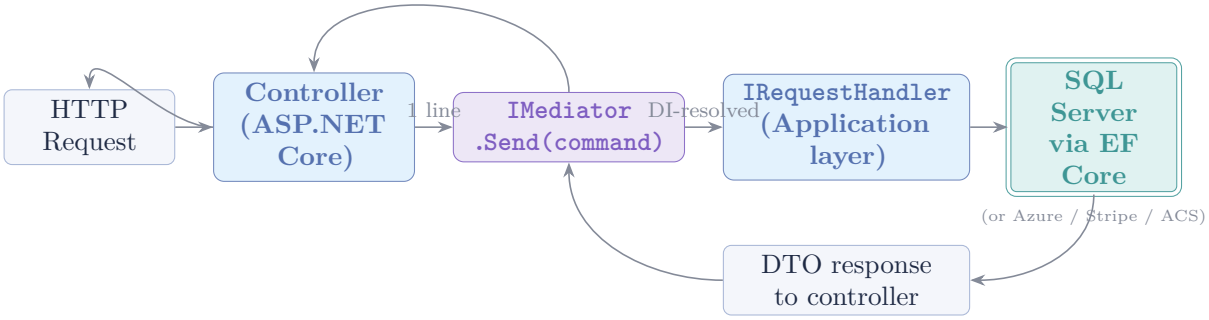

Figure 2. Every API call is one `mediator.Send()` — the controller is thin (1–3 lines). All business logic lives in the handler in the Application layer.

3.2 Example: AcceptBookingCommand

Listing 1. AcceptBookingCommandHandler (condensed)

```
[Auditable(AuditAction.BookingAccepted, "ConsultantBooking",
    TargetIdProperty = nameof(BookingId))]
public sealed record AcceptBookingCommand(Guid BookingId) : IRequest;

public sealed class AcceptBookingCommandHandler :
    IRequestHandler<AcceptBookingCommand>
{
    public async Task Handle(AcceptBookingCommand request, CancellationToken ct)
    {
        // 1. Load aggregate
        var booking = await context.Bookings
            .Include(b => b.Payment).FirstOrDefaultAsync(..., ct);

        // 2. Stripe capture
        await stripeService.CapturePaymentIntentAsync(...);

        // 3. Provision ACS video room
        var room = await meetingService.CreateRoomAsync(booking.Id, ct);
        booking.MeetingRoomId = room.RoomId;

        // 4. Persist & publish domain event
        await context.SaveChangesAsync(ct);
        await publisher.Publish(new BookingConfirmedEvent(...), ct);
    }
}
```

Why this design?

Why CQRS?

1. **Thin controllers** — controllers become routing-only; all business rules live in testable handler classes.
2. **Single responsibility** — each command or query has exactly one purpose. Adding a new feature means adding a new handler, not modifying an existing service.
3. **Audit by decoration** — `[Auditable]` is a MediatR pipeline behaviour: any command decorated with it automatically creates an `AuditLog` entry before the handler runs.
4. **Read/Write separation** — read queries bypass business-rule validation (no unnecessary `Includes` or domain logic).

3.3 Pipeline Behaviours (Cross-Cutting Concerns)

MediatR allows middleware-style behaviours that wrap every handler automatically:

Behaviour	Effect
`ValidationBehaviour`	Runs FluentValidation before the handler. Throws `ValidationException` on failure.
`AuditBehaviour`	Writes `AuditLog` row for commands marked `[Auditable]`.
`UnhandledExceptionBehaviour`	Catches all handler exceptions, logs them, re-throws.

4. Authentication & Security

Mechanism	Detail
Identity provider	ASP.NET Core Identity (users, roles, claims)
JWT algorithm	**RS256** (asymmetric) — private key signs, public key verifies. More secure than HS256: the public key can be shared with any verifier.
Access token TTL	15 minutes
Refresh token	Opaque token, rotated on every use (old token invalidated). Stored in HTTP-only cookie.
SSO	Google OAuth 2.0 and Microsoft (Entra ID) via `ExternalLoginInfo`
Roles	`SuperAdmin`, `Admin`, `Student`, `Consultant`, `Company`
2FA	TOTP (authenticator app) via ASP.NET Identity `TwoFactor` flow

Figure 3. Login flow. Access token travels in the Authorization header; refresh token is stored in an HTTP-only cookie (never accessible to JavaScript).

5. Real-Time Communication — SignalR

Three SignalR hubs provide real-time push to connected clients:

Hub	Events pushed	Used by
`ChatHub`	New message, typing indicator, read receipts	Community DMs
`NotificationHub`	Booking confirmed, payment processed, deadline reminder	All roles
`CommunityHub`	New post reactions, replies, online presence	Community feed

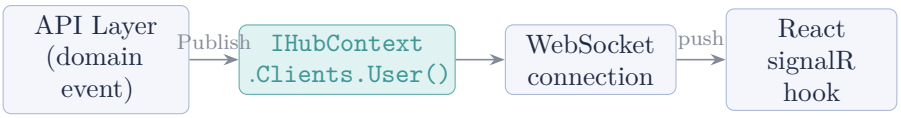

Figure 4. Real-time push: a domain event triggers `IHubContext.Clients.User(id).SendAsync(...)`. The client's `useSignalR` hook updates React state immediately.

6. Video Meeting — PB-006 (Azure Communication Services)

Key Point: What it is

A full in-browser video call between student and consultant, powered by **Azure Communication Services (ACS)**. The meeting is automatically created when a booking is confirmed and recorded to Azure Blob Storage.

6.1 Why Azure Communication Services?

Option	Pros	Cons & why rejected
Jitsi (self-hosted)	Free, open-source	Requires VM + ops; no built-in recording
Twilio	Mature, well-documented	Higher cost, US-centric data residency
ACS	Azure ecosystem, SLA, built-in recording, PII compliance	Slightly steeper SDK learning curve

6.2 Complete Video Meeting Flow (12 Steps)

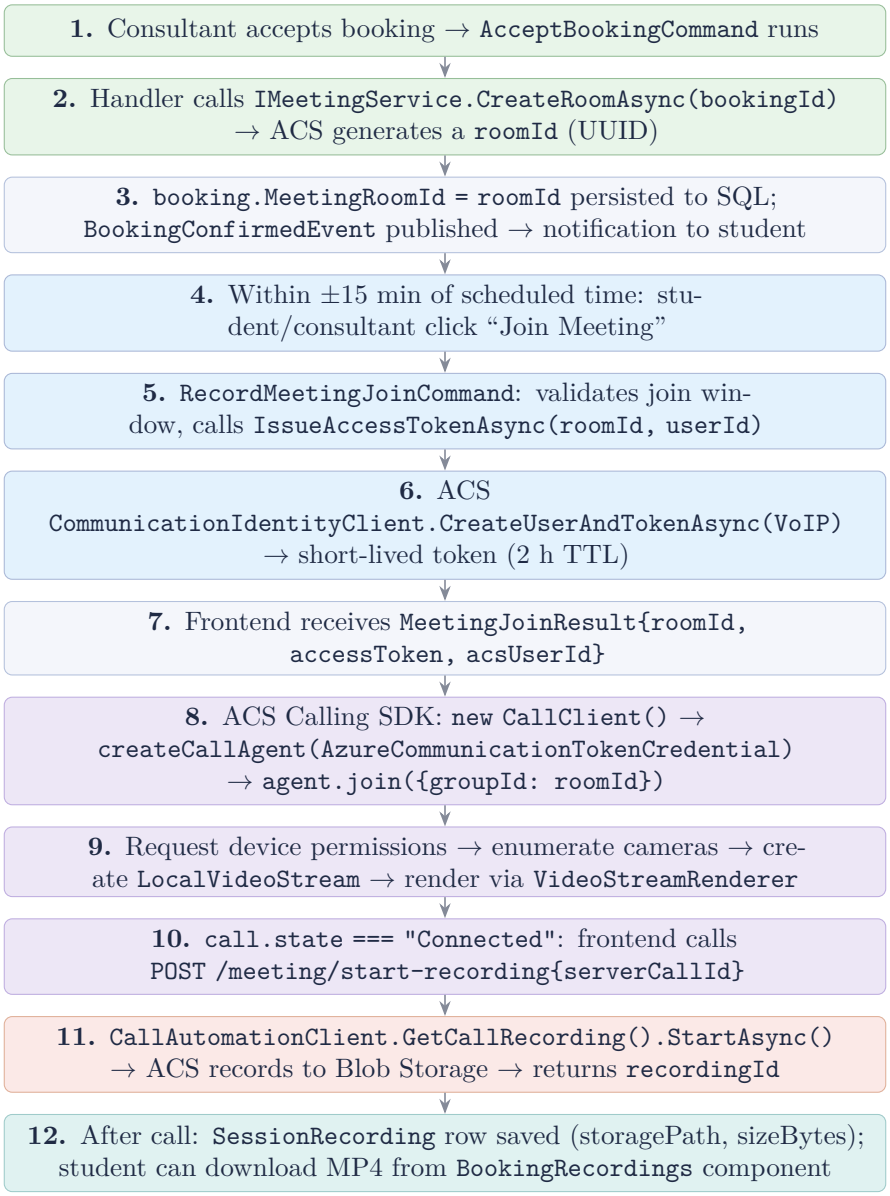

Figure 5. Complete video meeting lifecycle. Green = booking confirmation phase. Blue = join validation. Purple = ACS SDK in frontend. Orange = server-side recording. Teal = post-session storage.

6.3 Key Design Decisions

Decision	Rationale
Join window ±15 min	Prevents students from squatting in rooms hours before the session. Controlled by `JoinOpensBefore = JoinClosesAfter = TimeSpan.FromMinutes(15)`.
Lazy room provisioning	`MeetingRoomId` is set at booking acceptance, but if it is missing (legacy records), a new room is created at join time automatically.
Short-lived token (2 h)	ACS tokens expire after 2 h. For sessions exceeding this, the frontend would need to refresh — acceptable for a 60-min max session.
Stub implementation	`StubMeetingService` is injected in development. `join.provider === "Stub"` triggers a clear error message on the frontend: "Video not configured — contact admin."
Attendance tracking	`StudentJoinedAt` and `ConsultantJoinedAt` are set on first join. Used by the no-show detection job (FR-217).

6.4 Frontend — ACS Calling SDK Integration

Listing 2. Meeting.tsx — SDK bootstrap (condensed)

```
// 1. Get credentials from server
const join = await meetingsApi.join(bookingId);

// 2. Init ACS calling client + agent
const callClient = new CallClient();
const credential = new AzureCommunicationTokenCredential(join.accessToken);
const agent = await callClient.createCallAgent(credential, { displayName });

// 3. Device permissions + local camera
const deviceManager = await callClient.getDeviceManager();
await deviceManager.askDevicePermission({ video: true, audio: true });
const cameras = await deviceManager.getCameras();
const localStream = new LocalVideoStream(cameras[0]);

// 4. Join the group call
const call = agent.join(
  { groupId: join.roomId },
  { videoOptions: { localVideoStreams: [localStream] },
    audioOptions: { muted: false } },
);

// 5. Render local video
const view = await new VideoStreamRenderer(localStream).createView();
localVideoRef.current.replaceChildren(view.target);

// 6. Watch for remote participants
call.on("remoteParticipantsUpdated", (e) => {
  e.added.forEach(watchParticipant);
});

// 7. Start recording once connected
call.on("stateChanged", async () => {
  if (call.state === "Connected") {
    const serverCallId = await call.info.getServerCallId();
    await meetingsApi.startRecording(bookingId, serverCallId);
    setRecording(true);
  }
```

```
});
```

7. Background Jobs — Hangfire

Five recurring jobs run on the application server:

Job	Schedule	Action
KB Nightly Rebuild	Nightly 02:00 UTC	Incremental knowledge base rebuild
Deadline Reminder	Daily 08:00 UTC	Notifies students: 7-day + 1-day alerts
Payment Settlement	Every 15 min	Confirms captured Stripe payments
No-Show Detection	Hourly	Marks bookings where no participant joined
Analytics Sync	Nightly 03:00 UTC	Triggers ADF pipeline (Bronze refresh)

8. Analytics Data Pipeline (PB-015 – PB-018)

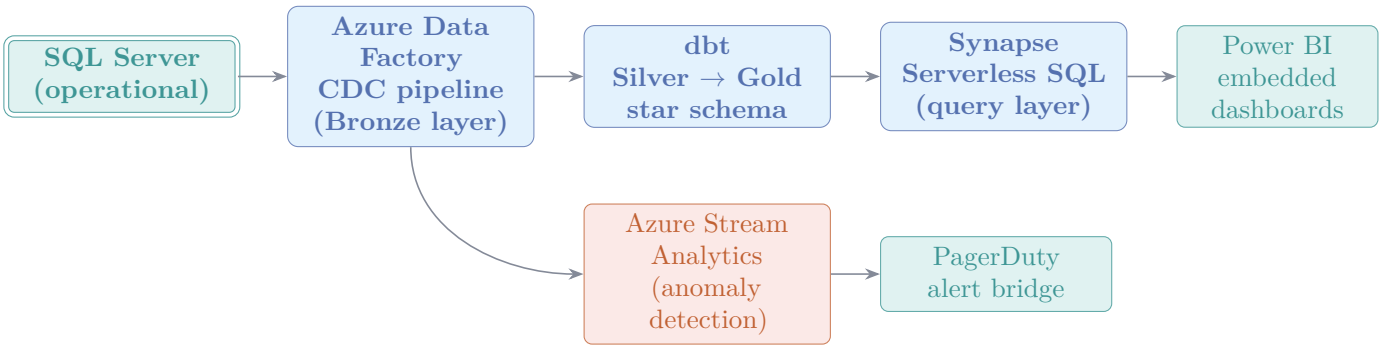

Figure 6. Analytics pipeline. CDC from SQL Server flows through ADF into the Bronze zone; dbt transforms to a Gold star schema (fact + dimension tables); Synapse Serverless exposes views to Power BI. A parallel Stream Analytics branch detects anomalies in real time and fires PagerDuty alerts.

9. Mahmoud's Modules — Deep Dive

As team lead and architect, Mahmoud owns 6 of the 18 PB epics:

PB	Module	Key deliverables
008	AI Features	RAG chatbot, recommendation scoring, eligibility check, KnowledgeBase indexer, fine-tuning dataset, admin KB management page, `AiInteraction` entity
011	Admin Portal	`AdminLayout` with 17-item navigation, admin dashboard (revenue widget, user growth chart), admin management pages for every entity
012	Audit & Compliance	`AuditBehaviour` (MediatR pipeline), `AuditLog` table, admin audit log viewer with full-text search, PII redaction sampling for AI chat responses
016	Data Warehouse	Azure Data Factory CDC pipelines (Bronze), dbt project (Silver & Gold star schema), Synapse Serverless views, reverse-ETL from Gold back to operational DB
017	AI Economy	Daily cost tracking per user & provider, `DailyCostLimitUsd` hard cap enforcement, recommendation click-through rate (CTR) tracking, admin AI cost dashboard
018	Real-time Streaming	Azure Stream Analytics job (payment anomalies + KB-rebuild triggers), PagerDuty webhook bridge, real-time Stripe event processing

9.1 PB-008 AI — Feature Ownership Summary

See companion document *06-ai-architecture-deep-dive.pdf* for full detail. Key summary:

- **Chatbot** (`AzureOpenAiService.AskAsync`): Standard RAG, 5-doc context, GPT-4o-mini, 3-level fallback to `LocalAiService`.
- **Recommendations** (`LocalAiService`): Deterministic scoring (Level 30 + Field 40 + Country 25 + Bonus 5), cached in `AiInteraction`, live re-hydration on read.
- **Eligibility** (`LocalAiService`): 3-criterion profile match → Eligible / PartiallyEligible / NotEligible verdict.
- **Admin KB page**: 4-button control panel (rebuild, force rebuild, import, JSONL export).

9.2 PB-012 Audit — How Auditing Works

Listing 3. is automatically audited]Any command decorated with [Auditable] is automatically audited

```
// In any Command record:
[Auditable(AuditAction.ScholarshipDeleted, "Scholarship",
    TargetIdProperty = nameof(ScholarshipId))]
public sealed record DeleteScholarshipCommand(Guid ScholarshipId) : IRequest;

// AuditBehaviour (registered as MediatR pipeline behaviour) intercepts it:
// 1. Reads [Auditable] attribute before handler runs
// 2. After handler succeeds: inserts AuditLog row
//    { UserId, Action, EntityType, EntityId, Timestamp, IpAddress }
// 3. Admin can query /api/admin/audit-logs with full-text search
```

10. Defence Presentation Flow

Defence Tip

This section gives the recommended order and talking points for presenting ScholarPath to the academic committee. Each block has a suggested time budget.

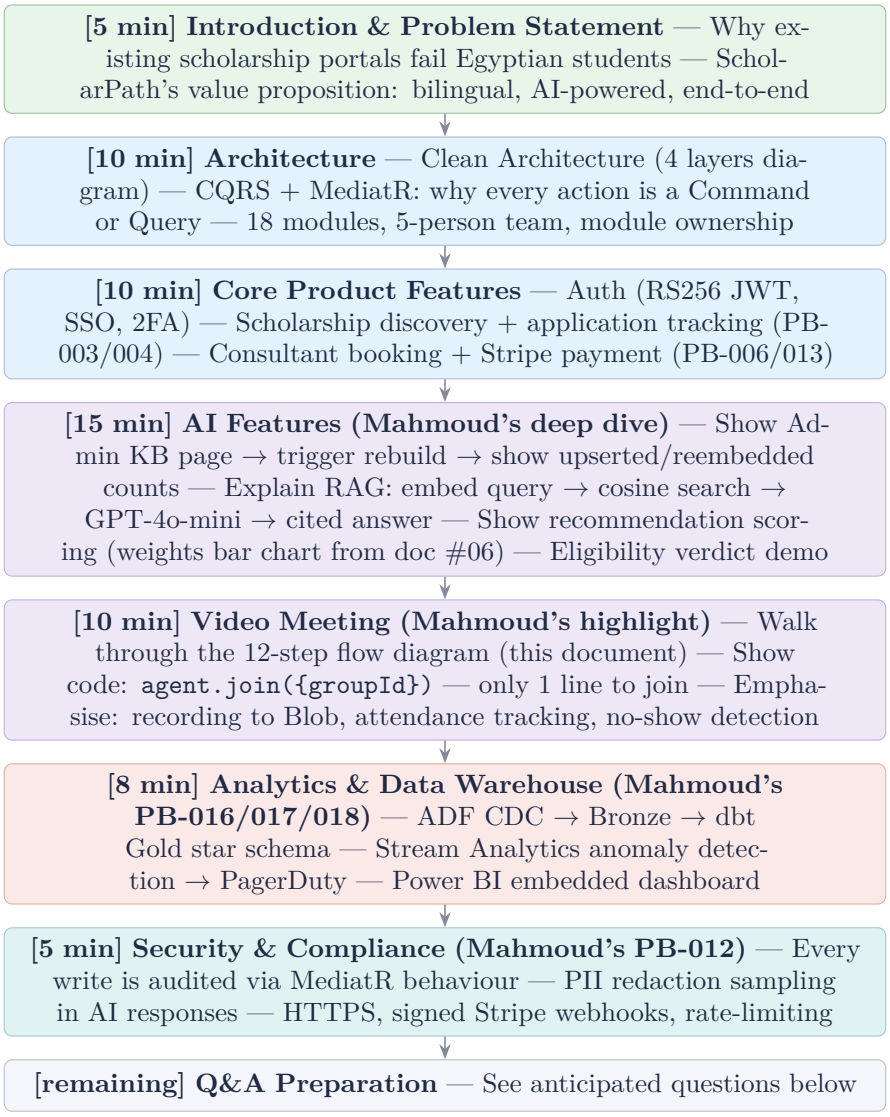

Figure 7. Recommended defence presentation order. Total: ≈63 minutes with buffer for questions between sections.

10.1 Anticipated Committee Questions — and Answers

Question	Answer
"Why Clean Architecture instead of a simple layered architecture?"	Clean Architecture enforces dependency inversion — the domain has zero knowledge of EF Core or Azure. This means we can swap the database (or any service) without touching business logic. We verified this by injecting a `Stub` implementation for all external services (ACS, Stripe, Azure) in tests.
"Why CQRS for a graduation project? Is it overkill?"	The project has 211 FRs across 5 developers. CQRS enforced clear ownership boundaries — each handler is a single C# file owned by one person. Without it, a shared service class would have caused merge conflicts on every sprint. MediatR also gave us cross-cutting audit behaviour for free.
"Why not use a vector database?"	At 692 documents, a full in-memory cosine scan completes in < 10 ms. A vector DB would add a network hop and operational cost with no accuracy benefit. The migration path exists: swap `KnowledgeRetriever` with a Qdrant or pgvector implementation.
"Is the AI hallucinating?"	The system prompt prohibits fabrication, and the answer must cite the numbered context block. The RAG context is grounded in the live database; if no document scores above 0.6, the local fallback gives a generic answer rather than guessing.
"Why Azure ACS for video and not Jitsi?"	Jitsi requires a self-hosted VM with operational overhead and has no built-in recording API. ACS provides a managed, PII-compliant recording pipeline and integrates natively with the rest of our Azure stack.
"How does billing work for consultants?"	Stripe PaymentIntents with manual capture. The student's card is authorised at booking time; capture happens only when the consultant accepts. Platform profit share is calculated at the handler level (Stripe Transfer).
"What happens if Azure OpenAI is down?"	Three-level fallback: (1) config missing → local; (2) HTTP error on embedding call → local; (3) GPT call fails → local. `LocalAiService.AskAsync` runs the same RAG retrieval without GPT, then falls back to a keyword router. Users always get a response.

11. Summary — What Makes ScholarPath Distinctive

Key Point: Three differentiators

1. Spec-driven development — every PR references an FR-xxx or US-xxx. Every entity has a migration. Every audit-relevant command has `[Auditable]`. 211 FRs fully implemented.

2. Production-grade AI at zero marginal cost — RAG chatbot + deterministic recommendations + eligibility checking, with a three-level fallback so the system degrades gracefully under any Azure failure. Daily cost cap prevents runaway billing.

3. End-to-end video infrastructure — ACS room provisioning, short-lived token issuance, client-side ACS Calling SDK, server-side recording, Blob storage, and MP4 download — all

wired together in 12 steps with attendance tracking and no-show detection.